Tema 6.

JSP

Servlets

J2EE



Java

JSPs

JDBC

# Indice

- Introducción
- Ciclo de Vida de JSP
- Elementos
  - Comentarios
  - Objetos implicitos
  - Directivas
  - Declaraciones
  - Scriptles
- Acciones
  - Principal acciones
  - Sesiones
  - JavaBeans
- Acceso a BD
- Modelo Vista Controlador

# 1. Introducción

- Servlets: Código **Java con lenguaje X/HTML embebido**
- Ventajas:
  - Uso de herramientas de diseño de páginas web directamente
  - Actualizaciones del aspecto gráfico no provocan el re-arranque del servidor
- Inconvenientes de Servlet Java:
  - Mezcla la capa de presentación y negocio.
  - Difícil mantenimiento

```
Collection customers = db.getCustomers();
PrintWriter writer = response.getWriter();
writer.println("<table border=\"1\">");
Iterator it = customers.iterator();
while(it.hasNext()) {
        writer.println("<tr>");
        writer.println("<td>");
        writer.println(((Customer)customers.next()).getCus tomerNumber());
        writer.println("</td>");
        writer.println("</tr>");
}
```

Azul: Capa de Negocio

Rojo: Capa de NO negocio

# 1. Introducción

- JSP (JavaServer Pages) es una especificación de Sun Microsystems, actualmente de Oracle
- Se utilizar para crear y gestionar páginas web dinámicas
- En los documentos JSP, se mezcla código HTML para generar la parte estática, con contenido dinámico creado a partir de marcas especiales
- El contenido dinámico **se obtiene, embebiendo dentro de la página código Java** de diferentes formas
- Su objetivo final es
  - separar la interfaz (presentación visual)
  - de la lógica

# 1. Introducción

- HolaMundo en JSP → Contexto = EjemplosClaseJSP
  - Se localiza en la parte pública de la aplicación → webapp/HolaMundo.jsp → Accesible
  - Se pueden colocar dentro de WEB-INF → Aquellas páginas JSP a las que no nos interesa que el usuario pueda acceder a ellas directamente → Accesible sólo mediante servlet o un redirección.
  - Se accede → http://localhost:8080/EjemplosClaseJSP/Holamundo.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8" import="java.util.Date"%>
<html>
<head>
  <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
  <title>Hola Mundo</title>
</head>
<body>
  <h1>¡Hola, Mundo!</h1>
  La fecha de hoy es: <%=new Date()%>
</body>
</html>
```

# 1. Introducción

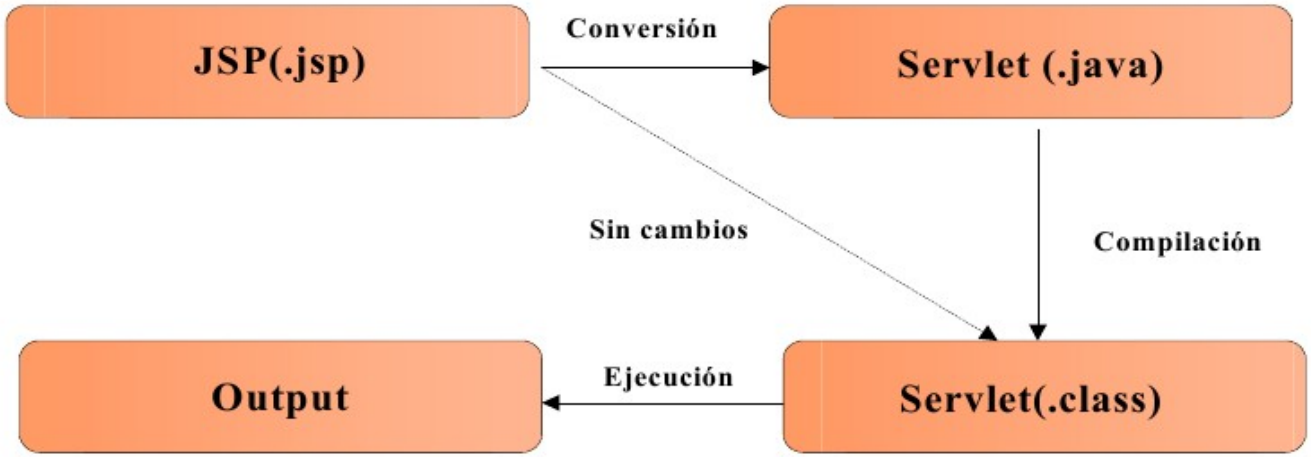
- Además del código X/HTML, el código fuente de una página JSP puede contener:
  - **Comentarios**
  - **Directivas**: Indican información general de la página, como puede ser importación de clases, página a invocar ante errores, si la página forma parte de una sesión, etc.
  - **Declaraciones**: Sirven para declarar métodos o variables.
  - **Scriptlets**: Código Java embebido.
  - **Expresiones**: Expresiones Java que se evalúan y se envían a la salida.
  - **Tags JSP**: Etiquetas especiales que interpreta el servidor.

# 2. Ciclo de vida de un JSP

- La página JSP se convierte en un servlet
- La conversión la realiza el servidor de aplicaciones la primera vez que se solicita la página JSP
- Una vez que se ha generado este servlet, cualquier otra petición para esa página JSP, se ejecuta el binario del servlet
- Si se modifica el código de la página JSP, entonces se regenera y recompila automáticamente el servlet y se recarga la próxima vez que sea solicitada
- Una ventaja muy importante de las páginas JSP: En servlets, cuando se modificaba un servlet, se tenía que recompilar y volver a desplegar manualmente el servlet → ahora es automático

# 2. Ciclo de vida de un JSP

- Ciclo de vida de un JSP:
  - Primera vez: a partir del JSP el servidor de aplicaciones crea un Servlet, lo compila (conversión)y lo carga en memoria.
  - En caso contrario: le pasa la petición al Servlet ya compilado.
  - Si se modifica el JSP desde la última compilación: se genera un nuevo Servlet, lo compila y lo vuelve a cargar en memoria



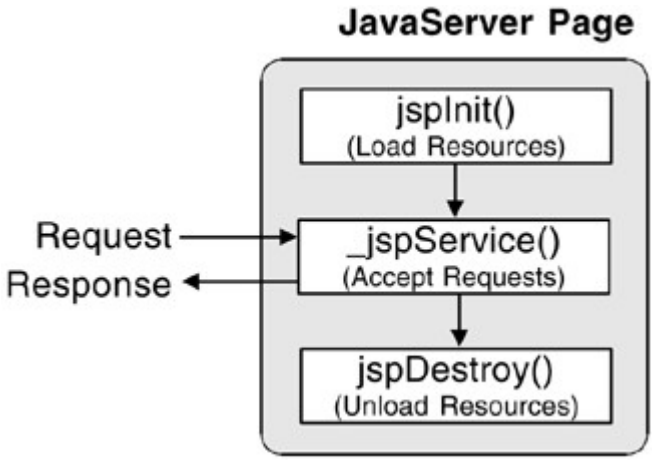
# 2. Ciclo de vida de un JSP

- Ciclo de vida de un JSP:
  - ✓ Durante el proceso de conversión, se genera una subclase de HttpServlet en cuyo método de servicio se incluye todo el contenido de la página, tanto el código Java que será incluido directamente en el método, como los bloques de texto HTML que serán insertados como cadenas de texto en el interior de métodos out.println()
  - ✓ Esta subclase de HttpServlet es específica de cada contenedor y debe implementar la interfaz HttpJspPage. Esta interfaz proporciona los métodos
    - jspInit(): Invocado por el método init
    - _jspService(): Invocado por service
    - JspDestroy(): Invocado por destroy() de la clase.
  - ✓ Es, por tanto, en el método de servicio _jspService() donde el contenedor incluye el código resultante de la transformación de la página en servlet.

# 2. Ciclo de vida de un JSP

- Se puede decir que una página JSP es una implementación de la clase javax.servlet.Servlet, que describe como crear un objeto (response) respuesta HttpServletResponse a partir de un objeto (request) petición HttpServletRequest.

```
import javax.servlet.jsp.*;
public final class hello_jsp extends HttpJspBase implements JspSourceDependent {
  private static java.util.Vector _jspx_dependants;
  public void _jspService(HttpServletRequest request, HttpServletResponse response)
       throws java.io.IOException, ServletException {
    JspFactory _jspxFactory = null;    PageContext pageContext = null;
    HttpSession session = null;    ServletContext application = null;
    ServletConfig config = null;    JspWriter out = null;
    Object page = this;    JspWriter _jspx_out = null;
    try {
      _jspxFactory = JspFactory.getDefaultFactory();
      response.setContentType("text/html");
      pageContext = _jspxFactory.getPageContext(this, request, response, null, true, 8192, true);
      application = pageContext.getServletContext();
      config = pageContext.getServletConfig();  session = pageContext.getSession();
      out = pageContext.getOut();
      _jspx_out = out;
      out.write("\n");
      out.write("<html>\n    ");      out.write("<head>\n      ");
      out.write("<title>Hello JSP");      out.write("</title>\n    ");
      out.write("</head>\n    ");      out.write("<body>\n      ");
      out.write("<h1>Hello World");      out.write("</h1>\n      The time is ");
      out.write(String.valueOf( new java.util.Date() ));      out.write("\n    ");
      out.write("</body>\n");      out.write("</html>\n");
      ...
```



# 2. Ciclo de vida de un JSP

- JSP → Archivo compuesto por:
  - **Comentario <%-- comentario --%>**
  - **Objetos implicitos**
  - **Scriptlet**: código servidor Java, escrito entre **<%** codigo Java **%>**
  - **Expresión**: inserta valores Java directamente en la salida **<%=** expresión Java **%>**
  - **Declaración**: define métodos o campos que son insertados dentro de la clase Servlet **<%!** codigo Java **%>**
  - **Directiva**: dos tipos principales “page” e “include” **<%@** directiva atributo=”valor” **%>**
    - ✓ <%@ page import=”clase” %>
    - ✓ <%@ include file=”fichero” %>
  - **Tag**: instrucción en formato XML asociada a clases Java

# 3. Elementos

- Se pueden establecer tres tipos de comentarios
  - Comentarios html <!--Comentario-->
    - ✓ Se envía como respuesta aunque no se renderiza (como todos los comentarios HTML). Estos comentarios también serán incluidos dentro del código del Servlet generado
  - Comentario JSP <%-- Comentario de página JSP-- %>
    - ✓ Sólo están en las páginas JSP. No se incluyen en el Servlet durante la fase de traducción, ni se envían al navegador del usuario.
  - Comentarios Java //
    - ✓ Estos comentarios se traducen al código fuente del Servlet que se generará a partir de la página JSP, pero no se envían al usuario cuando realiza una petición.

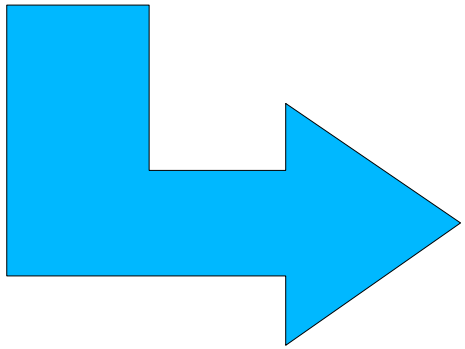
# 3. Elementos

- JSP tiene un conjunto **objetos implícitos**, basados en la API de servlets. Estos objetos
  - **request** (HttpServletRequest): Petición
  - **response** (HttpServletResponse): Respuesta
  - **out** (PrintWriter): Flujo de salida
  - **session** (HttpSession): Traza del usuario
  - **application** (ServletContext): Es el ServletContext obtenido mediante getServletConfig().getContext().
  - **config** (ServletConfig): El objeto ServletConfig para esta página.
  - **pageContext** (PageContext): JSP presenta una nueva clase llamada PageContext para encapsular características de uso específicas del servidor como JspWriters de alto rendimiento.
  - **page** (this): Sinónimo de this,

# 3. Elementos

- Objetos implicitos: HelloWorldName.html

```
<html>
<head>
  <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
  <title>HelloWorldName</title>
</head>
<body>
    <h1>Formulario de petición de nombre</h1>
    <!-- Se envía al JSP “HelloWorldName.jsp” -->
   <form method="post" action="HelloWorldName.jsp">
        <p>Introduce tu nombre: <input type="text" name="nombre"></p>
        <p><input type="submit" value="Enviar"></p>
   </form>
</body>
</html>
```





# 3. Elementos

- Objetos implicitos: HelloWorldName.jsp

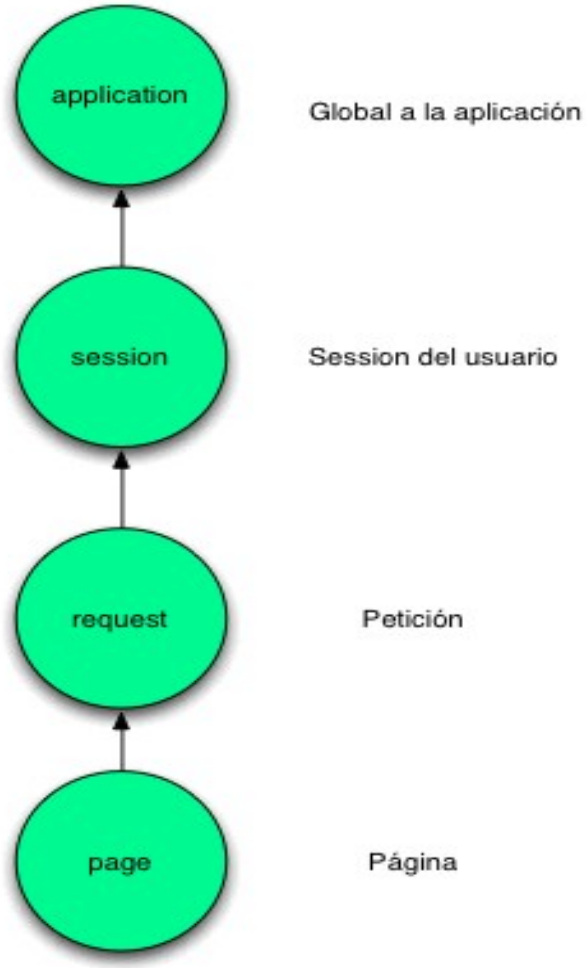
```
<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8"%>
<html>
   <head>
         <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
         <title>HelloWorldName</title>
   </head>
   <body>
        <h1>Saludo al cliente</h1>
        <%
        String nombre = request.getParameter("nombre");
        out.println("Tu nombre es: " + nombre);
        %>
   </body>
</html>
```

Captura del parámetro

Flujo de salida

# 3. Elementos

- JSP tiene un conjunto **objetos implícitos**, basados en la API de servlets.



# 3. Elementos

- Objetos implicitos → application
  - Contador de visitas

Objeto application → recupero atributo contador del contexto

```
<%
 Integer contador = (Integer) application.getAttribute("contador");
 if (contador == null) {
        contador = new Integer(0);
 }

  application.setAttribute("contador", new Integer(contador.intValue() + 1));

%>
<h1><%=contador%> visitas </h1>
```

Almaceno el nuevo valor del atributo

# 3. Elementos

- Declaraciones de variables y objetos

<%! declaration; [ declaration; ]+ ... %>
o
<jsp:declaration>
code fragment [ declaration; ]+ ...
</jsp:declaration

Ejemplo
<%! int cont = 0; %>
<%! String s; %>
<%! Persona p = new Persona(“Luis”); %>

- Expresiones: expresión que será evaluada y **el resultado será establecido** en la salida.

<%= expression %>
o
<jsp:expression>
expression
</jsp:expression>

Ejemplo
<%= cont +1; %>
<%= x+y+5; %>
<%= persona.toString(); %>

- Scriptlets. Código java que se incluirá en jspservice().

<% código %>
o
<jsp:scriptlet>
código
</jsp:scriptlet>

Ejemplo

```
<%
String nombre = request.getParameter("nombre");
out.println("Tu nombre es: " + nombre);
%>
```

# 3. Elementos

- Ejemplo: HelloWorldName2.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8"%>
<%!int contar = 0;%>
<%!public String mostrarNombre(String name) {
        if (name.equals("Luis")) {
                return ("Tu nombre es " + name);
        } else {
                return ("Tu nombre no es Luis");
        }
}%>
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <title>HelloWorldName</title>
  </head>
  <body>
        <h1>Hola Mundo</h1>
        <%
                String nombre = request.getParameter("nombre");
                out.println(mostrarNombre(nombre));
                contar += 1;
        %>
        <h1>Eres el visitante<%=contar%></h1>
   </body>
</html>
```

Declaraciones

Scriptles

Expresión

# 3. Elementos

- Directivas: Mensajes al contenedor de Servlets

```
<%@ directiva [atributo= "valor"]* % >
```

  - Include: Permite añadir un fichero HTML / JSP en la salida. Suele ser contenido repetitivo que se separa en ficheros independientes (se hace en el proceso de transformación).

```
<%@ include file="cabecera.jsp" %>
```

  - Page: Permite configurar la página

```
<%@ page ATRIBUTOS %>
```

```
<%@ page import=”java.util.*” %>
```

# 3. Elementos

- Directivas: Mensajes al contenedor de Servlets
  - Page: Algunos de los atributos más usados son:
    - import: Lista de paquetes o clases, separados por comas a incluir
    - contentType: Especifica el tipo MIME de los contenidos que van a ser generados (por defecto HTML)
    - pageEncoding: Codificación de caracteres de la página JSP (por defecto utf-8)
    - ErrorPage: Referencia a una página que se muestra si se produce alguna excepción y no se captura en la propia página
    - Session: Indica si una JSP puede participar en la sesión del usuario, y por tanto acceder a su contenido o almacenar nuevo contenido en ella.
  - TagLib
    - Permite extender JSP con etiquetas o marcas generadas por el propio usuario (etiquetas personalizadas)
    - Se hace referencia a una biblioteca de etiquetas que contiene código Java compilado que establece el comportamiento de las etiquetas

# 3. Elementos

- Directivas: EjemploDirectiva.jsp

```
<%@ page
autoFlush="true" buffer="16kb" contentType="text/html"
errorPage="myErrorPage.jsp" extends="org.apache.jasper.runtime.HttpJspBase"
import="java.sql.*,java.util.*" info="Página de ejemplo de directiva"
language="java" pageEncoding="UTF-8" session="false"%>

<HTML>
  <HEAD>
        <TITLE>Página con directirva</TITLE>
  </HEAD>
  <BODY>
        <%@ include file="cabecera.html"%>
        <%@ include file="menu.jsp"%>
        <div id="content">
            <%
                String nombre = request.getParameter("nombre");
                out.println("Tu nombre es: " + nombre);
            %>
        </div>
  </BODY>
</HTML>
```

Directiva

Include

Scriptles

# 3. Elementos

## Ejemplo AdivinarNúmero (obtenido de [1])

```
<%! int adivinar = (int) (java.lang.Math.random() * 100)+1;%>
<%! int intentos = 0;%>
<%! public String ayuda(int numero) {
        if (numero > adivinar) { return "El numero es menor";}
        return "El numero es  mayor";
    }%>
<html>
    <head><title>Adivinar el numero</title></head>
    <body>
        <% String parametro = request.getParameter("adivinar");
           int numero;
           if (parametro != null) {
                numero = Integer.parseInt(parametro); intentos++;
           }
           else { numero = -1; }
           if (parametro != null && adivinar == numero) {%>
              <p> ¡Enhorabuena! Lo has adivinado despues de <%= intentos%> intentos.</p>
              <% adivinar = (int) (java.lang.Math.random() * 100)+1;
               intentos = 0;
               %>
              ¿Quieres probar otra vez <a href="/EjemplosClaseJSP/AdivinarNumeros.jsp"></a>?
           <% }
           else if (intentos == 0) {%> <p>Adivina el numero; esta entre uno y 100:</p> <% }
           else {%>
            <p>No es ese.  <%= ayuda(numero)%>.</p>
            <p>Has hecho un total de <%= intentos%> intentos.</p>
        <% }%>
        <form method=get>
            ¿Cual es el numero? <input type=text name="adivinar">
            <input type=submit value="Submit">
        </form>
    </body>
</html>
```

Declaraciones

Parámetro de entrada

Recupero número

Si es el número

Se vuelve a lanzar la jsp

# Ejercicio 1.

- Se pide realizar un programa que calcule el factorial de un número usando JSP. Se debe proporcionar cuatro soluciones:
  - Solución 1: usando únicamente JSP (el número es recibido por parámetro. Sino se recibe el número, se muestra un formulario.

```
public long Factorial(int numero) {
  long r = 1;
  for (int i = 1; i <= numero; i++) {r = r * i;}
  return r;
}
```

# 4. Acciones

- **Acciones**:
  - Son etiquetas, con formato XML, que afectan al comportamiento en tiempo de ejecución del JSP
  - Durante el proceso de traducción de JSP al servlet, la marca se reemplaza por cierto código Java asociado a la etiqueta
  - Su objetivo es reducir la cantidad de código java necesario dentro una página JSP
  - Se puede considerar como una macro
  - Existen un conjunto de acciones estándar predefinidas, pero los usuarios pueden definir las suyas propias utilizando las bibliotecas de tags (taglibs)
  - Sintaxis
    - ✓ <prefijo:sufijo atributos />
  - Ejemplo
    - ✓ <jsp:include page="mi_jsp.jsp" />

# 4. Acciones

- **Acciones**: Algunas acciones son:
  - <**jsp:attribute name**=”nombre”>
  - <**jsp:body**>
  - <**jsp:element name**=”nombre”>
  - <**jsp:include page**=”url”>
  - <**jsp:forward page**=”url”>
  - <**jsp:param name**=”nombre” **value**=”valor”>
  - <**jsp:text**>
  - <**jsp:useBean id**=”nombre” **class**=”clase” />
  - <**jsp:setProperty name**=”idBean” **property**=”prop” **value**=”valor”>
  - <**jsp:getProperty name**=”idBean” **property**=”prop”>

# 4. Acciones

- **<jsp:include page=**”url”>
  - Permite incluir un recurso establecido en la URL, en la petición JSP en tiempo de ejecución
  - En el proceso de traducción, en el método _jspService() se genera el código que incluye el recurso
  - **IMPORTANTE**: Cuando se ejecuta, se invoca al mecanismo que incluye el recurso y el resultado se añade a la página
  - El elemento incluido puede acceder al objeto request de la página padre, y además de los parámetros normales, a los que se añadan con <jsp:param>
- **Diferencia** entre la directiva include y acción jsp_include
  - Directiva: se incluye la información del fichero en el momento de hacer la traducción a servlet (momento inicial)
  - Acción <jsp:include> no se añade código al servlet, sino que se invoca al código en tiempo de ejecución (momento de la petición) (cuando el include depende de los parámetros de entrada)

# 4. Acciones

## <jsp:include page="url">

```
<%@ page language="java" contentType="text/html;
charset=UTF-8"
pageEncoding="UTF-8"%>
<h1>Hola Mundo</h1>
<%
String nombre = request.getParameter("nombre");
out.println("Tu nombre es: " + nombre);
%>
```

HelloWorldName.jsp
(Sin HTML)

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8"%>
<%!int contar = 0;%>
<%!public String mostrarNombre(String name) {
if (name.equals("Luis")) {
return ("Tu nombre es " + name);
} else {
return ("Tu nombre no es Luis");
}
}%>
<h1>Hola Mundo</h1>
<%
String nombre = request.getParameter("nombre");
out.println(mostrarNombre(nombre));
contar += 1;
%>
<h1>
Eres el visitante<%=contar%></h1>
```

HelloWorldName2.jsp
(Sin HTML)

HelloWorld_Acciones.html

```
<body>

<h1>Formulario de petición de nombre</h1>
<!-- Se envía el formulario al JSP "saludo.jsp" -->
<form method="post" action="Accion_JSP_Include.jsp">
<p>
Introduce tu nombre: <input type="text" name="nombre">
</p>
<p>
<input type="submit" value="Enviar">
</p>
</form>
</body>
```

Accion_JSP_Include.jsp

```
<%@ page language="java" contentType="text/html;
charset=UTF-8"
pageEncoding="UTF-8"%>
<html>
<head>
<title>HelloWorldName</title>
</head>
<body>
 <jsp:include page="HelloWorldName.jsp" />
 <jsp:include page="HelloWorldName2.jsp" />
</body>
</html>
```

# 4. Acciones

- <**jsp:param name**=”nombre” **value**=”valor”>
  - Permite añadir parámetros en la llamada que se realiza

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8"%>
<html>
<head>
<title>HelloWorldName</title>
</head>
<body>
 <jsp:include page="HelloWorldName.jsp" >
       <jsp:param name="nombre" password="prueba" />
 </jsp:include>
 <jsp:include page="HelloWorldName2.jsp" />
</body>
</html>
```

# 4. Acciones

- <**jsp:forward page**=”url”>
  - Permite reenviar una petición a otro recurso (HTML o JSP)
  - Se pueden adjuntar nuevo parámetros, mediante jsp:param
  - Una vez realizado el reenvio, no se vuelve al servlet original

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8"%>
<html>
<head>
<title>HelloWorldName</title>
</head>
<body>
 <jsp:forward page="HelloWorldName2.jsp" />
       <jsp:param name="nombre" password="prueba" />
 </jsp:forward>
</body>
</html>
```

# Ejercicio 1.

- Se pide realizar un programa que calcule el factorial de un número usando JSP. Se debe proporcionar cuatro soluciones:
  - Solución usando únicamente JSP (el número es recibido por parámetro.
  - Solución usando dos JSP. En un JSP (calcular.jsp) se realiza el cálculo y en el otro JSP (Factorial2.jsp) se incluye con la acción jsp:include.
  - Solución usando dos JSP: Factorial3.jsp calcula el JSP y lo muestra y error.jsp que muestra cualquier error en el cálculo (con jsp:forward).

```
public long Factorial(int numero) {
  long r = 1;
  for (int i = 1; i <= numero; i++) {r = r * i;}
  return r;
}
```

# 4. Acciones

- <**jsp:useBean id**=”nombre” **class**=”clase” />

  - Sirve para instanciar un JavaBean (objeto de una clase). Si no existe la crea, y si existe utiliza una instancia ya existente
  - Qué es un JavaBeans?? Son objetos Java que cumplen ciertas características
    - ✓ Por defecto: son serializables y tienen un constructor implícito (sin parámetros)
    - ✓ Obligatoriamente tiene que tener setAttributo() getAtributo() de todos los atributos
  - Esta etiqueta crea un JavaBean de una clase determinada. Es el equivalente a utilizar new en Java

**<jsp:useBean id=“p” class=”Persona/> ↔ p= new Persona()**

# 4. Acciones

- <**jsp:useBean id**=”nombre” **class**=”clase” **scope=**”???”/>
- Los JavaBeans se almacenan dentro un ámbito (scope), pudiendo ser:
  - page: el ámbito será la propia página JSP. Cuando el control sea transferido a otra página JSP, u otro Servlet, esta variable desaparecerá (valor por defecto)
  - request: el ámbito será la petición. Cuando finalice la petición, se destruirá el objeto.
  - session: el ámbito será la sesión del usuario. Para que una página JSP pueda almacenar y recuperar información de la sesión del usuario el valor del atributo session de la directiva page debe ser "true" (el valor por defecto): <%@ page session= "true" %>.
  - application: el ámbito será el contexto de la aplicación.

# 4. Acciones

- <**jsp:useBean id**=”nombre” **class**=”clase” />
  - **jsp:getProperty** y **jsp:getProperty** permite acceder a las propiedades/atributos de un bean (accesibles mediante get y set)
    - <jsp:setProperty name="p1" property="nombre" value="Madrid" /> es equivalente a p1.setNombre( "Madrid" ).
    - <jsp:getProperty name="p1" property="edad" es equivalente a p1.getEdad()
  - El atributo property puede contener los siguientes valores
    - Property=“*”: Se capturan como propiedades y valores todos los parámetros del objeto request
    - property=“Nombre”: El valor se coge de un parámetro con el mismo nombre del objeto request
    - property=“Nombre” param=“NombreParámetro” El valor de la propiedad Nombre se saca del valor del parámetro NombreParámetro
    - property=“Nombre” value=“ValorParámetro” Se asigna el valor valorParámetro

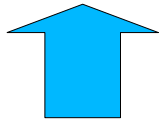
# 4. Acciones

## Ejemplo de Bean

### Clase HolaMundo.java

```
package es.unex.cum.tw;
public class HolaMundo{
   private String nombre;
   public String getNombre(){
          return nombre;
   }
   public void setNombre(String n){
          nombre=n;
   }
}
```

http://localhost:8080/EjemplosClaseJSP/Bean/HolaMundoBean.jsp



### HolaMundoBean.jsp

Declaración de un Objeto p → Ambito página

Se establece su valor

Valor fijo

Posteriormente se recupera

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8" errorPage="error.jsp"%>

<jsp:useBean id="p" scope="page" class="es.unex.cum.tw.bean.HolaMundo"/>
<html>
  <head>
          <title>Hola Mundo con Bean</title>
  </head>
  <body>
     <jsp:setProperty property="nombre" name="p" value="Luis" />
     <h3> El nombre almacenado en el Bean es:
          <jsp:getProperty property="nombre" name="p" />
     </h3>
   </body>
</html>
```

# 4. Acciones

- Ejemplo de Bean → Cuidado: recibiendo el valor por parámetro

Clase HolaMundo.java

```
package es.unex.cum.tw;
public class HolaMundo{
   private String nombre;
   public String getNombre(){
          return nombre;
   }
   public void setNombre(String n){
          nombre=n;
   }
}
```

http://localhost:8080/EjemplosClaseJSP/Bean/HolaMundoBean.jsp?name=Luis

HolaMundoBean.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8" errorPage="error.jsp"%>

<jsp:useBean id="p" scope="page" class="es.unex.cum.tw.beans.HolaMundo"/>
<html>
  <head>
          <title>Hola Mundo con Bean</title>
  </head>
  <body>
     <jsp:setProperty property="nombre" name="p" param="nombre" />
     <h3> El nombre almacenado en el Bean es:
          <jsp:getProperty property="nombre" name="p" />
     </h3>
   </body>
</html>
```

Recuperación Parámetro name

## Ejemplo de Bean: Calculadora (1/2)

```
package es.unex.cum.tw.bean;

public class Calculadora {
  private int op1;
  private int op2;
  public int getOp1() {
         return op1;
  }
  public void setOp1(int op1) {
         this.op1 = op1;
  }
  public int getOp2() {
         return op2;
  }
  public void setOp2(int op2) {
         this.op2 = op2;
  }
  public int getSuma(){
         return op1+op2;
  }
  public int getResta(){
         return op1-op2;
  }
}
```

## Ejemplo de Bean: Calculadora (2/2)

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8"%>

   <jsp:useBean id="c" scope="page" class="es.unex.cum.tw.bean.Calculadora" />
<html>
 <head>
         <title>Ejemplo Calculadora</title>
 </head>
 <body>
   <jsp:setProperty property="op1" name="c"
         value="<%=Integer.parseInt(request.getParameter(\"ope1\"))%>" />
   <jsp:setProperty property="op2" name="c"
         value="<%=Integer.parseInt(request.getParameter(\"ope2\"))%>" />

   <h3>El resultado de la suma es:
         <jsp:getProperty property="suma" name="c" />
   </h3>
 </body>
</html>
```

Captura de op1 a partir del parámetro ope1

Llamada al método suma de Calculadora

Llamada: Necesita el parámetro ope1 y ope2

http://localhost:8080/EjemplosClaseJSP/Bean/Calculadora.jsp?ope1=5&ope2=15

# 4. Acciones

- Ejemplo de Bean: Calculadora
  - jsp:setProperty puede guardar en un Java Bean los parámetros de un formulario que han sido enviados a una página JSP directamente → Durante la declaración del java bean
    - ✓ En este caso el nombre de los parámetros deben llamarse igual
    - ✓ Si fueran distintos
      - <jsp:setProperty property="op1" name="c" param="ope1"/>

Captura automática

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8"%>
<jsp:useBean id="c" scope="page" class="es.unex.cum.tw.bean.Calculadora">
        <jsp:setProperty property="*" name="c" />
</jsp:useBean>
<html>
  <head><title>Ejemplo Calculadora</title></head>
  <body>
    <h3> El resultado de la suma es:
          <jsp:getProperty property="suma" name="c" />
    </h3>
  </body>
</html>
```

Llamada: http://localhost:8080/EjemplosClaseJSP/Bean/Calculadora.jsp?op1=5&op2=15

## Ejemplo de Bean: Adivinar un número (AdivinarNumero.java)

```
package es.unex.cum.tw.bean;
import java.util.Random;

public class AdivinarNumero {

  int solucion;
  boolean acierto;
  String ayuda;
  int intentos;

public AdivinarNumero() {
   Resetear();
}

public boolean getAcierto() {
   return acierto;
}

public String getAyuda() {
   return "" + ayuda;
}

public int getIntentos() {
   return intentos;
}
```

```
public void setProbar(String valor) {
  intentos++;
  int g;
  try {
          g = Integer.parseInt(valor);
   } catch (NumberFormatException e) {
          g = -1;
   }

  if (g == solucion) {
          acierto = true;
  } else if (g == -1) {
          ayuda = "Debe ser un número";
  } else if (g < solucion) {
          ayuda = "Es mayor";
  } else if (g > solucion) {
          ayuda = "Es menor";
  }
}
public resetear() {
   solucion=
      Math.abs(new Random().nextInt() %
100) + 1;
   acierto = false;
   intentos = 0;
}
}
```

# 4. Acciones

- Ejemplo de Bean: Adivinar un número (AdivinarNumero.jsp)

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
<jsp:useBean id="numero" class="es.unex.cum.tw.bean.AdivinarNumero" scope="session"/>
<jsp:setProperty name="numero" property="*" />
<html>
<head><title>Juego de la adivinanza</title></head>
<body>
<%  if (numero.getAcierto()) {%>
   <p>Prueba conseguida tras:
   <%=numero.getIntentos()%></p>
   <%numero.resetear();%>
   <a href="AdivinarNumero.jsp">Otra vez</a>?
<%} else if (numero.getIntentos() == 0) { %>
   <p>Bienvenido a este juego</p>
   <p>Piensa un número del 0 al 100</p>
   <form method=get>
         Introduce un número<input type=text name=probar />
        <input type=submit value="Submit" />
   </form>
<%} else { %>
   <p>Lo siento pero no.</p>
   <b><%=numero.getAyuda()%></b> Llevas:
   <%=numero.getIntentos()%>
   <form method=get>
         Intentalo otra vez <input type=text name=probar>
         <input type=submit value="Submit" />
   </form>
<%
}
%>
</body>
</html>
```

JavaBean a nivel de Sesión. La 1º vez lo crea

Coge parámetro probar

Si has acertado

Si es el primer intento

Si es el segundo

# Ejercicio 1.

- Se pide realizar un programa que calcule el factorial de un número usando JSP. Se debe proporcionar cuatro soluciones:
  - Solución usando únicamente JSP (el número es recibido por parámetro.
  - Solución usando dos JSP. En un JSP (calcular.jsp) se realiza el cálculo y en el otro JSP (Factorial2.jsp) se incluye con la acción jsp:include.
  - Solución usando dos JSP: Factorial3.jsp calcula el JSP y lo muestra y error.jsp que muestra cualquier error en el cálculo (con jsp:forward).
  - Solución usando JavaBean

```
public long Factorial(int numero) {
  long r = 1;
  for (int i = 1; i <= numero; i++) {r = r * i;}
  return r;
}
```

# 5. JSP y Bases de Datos

- Al igual que con Servlet, se puede utilizar en las páginas JSP tanto como JDBC como Pool. Ejemplo de JDBC

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
<%@ page import="java.sql.Connection"%><%@ page import="java.sql.Statement"%>
<%@ page import="java.sql.ResultSet"%><%@ page import="java.sql.SQLException"%>
<%@ page import="java.sql.DriverManager"%>
<%! Connection con=null; %>
<%  try {
    Class.forName("com.mysql.jdbc.Driver");
    String URL = "jdbc:mysql://localhost:3306/tw";
    con = DriverManager.getConnection(URL,"tw","tw2425");
    Statement sentencia = con.createStatement();
    ResultSet rs=null;
    synchronized (sentencia) {
         rs = sentencia.executeQuery("select * from productos");
     }
     out.println("<table border='1'>");
     out.println("<tr><td><b>Producto</b></td><td><b>Comentarios</b></td></tr>");
    while (rs.next()) {
         out.println("<tr><td>”+rs.getString("nombreProd"));
         out.println("</td><td>" + rs.getString("comentarios"));
    }
    out.println("</td></tr>");
  } catch (SQLException e) {e.printStackTrace();
  } catch (ClassNotFoundException cnfe) {e.printStackTrace();}
  out.println("</TABLE></P>");
  con.close();
%>
```

# 5. JSP y Bases de Datos

- Al igual que con Servlet, se puede utilizar en las páginas JSP tanto como JDBC como Pool. Ejemplo de Pool

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
<%@ page import="java.sql.Connection"%><%@ page import="java.sql.Statement"%>
<%@ page import="java.sql.ResultSet"%><%@ page import="java.sql.SQLException"%>
<%@ page import="javax.naming.Context"%>
<%@ page import="javax.naming.InitialContext"%>
<%@ page import="javax.sql.DataSource"%>
<% Context ctx = new InitialContext();
   DataSource ds = (DataSource) ctx.lookup("java:comp/env/jdbc/testdb");
   Connection con = ds.getConnection();
   Statement sentencia = con.createStatement();
   ResultSet rs=null;
   synchronized (sentencia) {
         rs = sentencia.executeQuery("select * from productos");
   }
   out.println("<table border='1'>");
   out.println("<tr><td><b>Producto</b></td><td><b>Comentarios</b></td></tr>");
  try {
     while (rs.next()) {
         out.println("<tr><td>+ rs.getString("nombreProd"));
         out.println("</td><td>" + rs.getString("comentarios"));
      }
      out.println("</td></tr>");
   } catch (SQLException e) { e.printStackTrace(); }
   out.println("</TABLE></P>");
   con.close();
%>
```

# 5. JSP y Bases de Datos

- Lo habitual es que el acceso a las bases de datos no se realiza en las páginas JSP, sino que este proceso lo realicen los javaBeans.
- De este modo se consigue una separación completa de cada una de las capas: Lógica (Beans) y la presentación (JSP) junto con los servlets actuando como Controladores, lo que ha dado lugar al Módelo Vista Controlador (MVC)

# Ejercicio 2.

- En la sesión 6
  - Solo había un único usuario
  - En la sesión se almacenaba la lista de regalos

| Tipo de archivo | Nombre | |
|---|---|---|
| css | WebContent/css/CSS.css | Css del ejercicio |
| html | Autenticar.html | Página para introducir el usuario y contraseña |
| | Error.html | Página cuando se produce un error |
| | Inicio.html | Página principal del proyecto |
| | Introducir.html | Página para añadir regalos a la carta |
| Servlet | Action.java | Servlet que dependiendo de la acción (Añadir o Listar) redirecciona la acción |
| | Anadir.java | Servlet para añadir un nuevo regalo |
| | Listar.java | Servlet para listar la bolsa de regalo |
| | Login.java | Servlet para la autenticación (inicio de sesión) |
| | Logout.java | Servlet para anular la sesión |

# Ejercicio 2.

- En la sesión 7:
  - Había distintos usuarios alojados en una BD
  - Cada usuario tenía su carta a los reyes magos.

| Tipo de archivo | Nombre | |
|---|---|---|
| css | WebContent/css/CSS.css | Css del ejercicio |
| html | Autenticar.html | Página para introducir el usuario y contraseña |
| | Error.html | Página cuando se produce un error |
| | Inicio.html | Página principal del proyecto |
| | Introducir.html | Página para añadir regalos a la carta |
| Servlet | Action.java | Servlet que dependiendo de la acción (Añadir o Listar) redirecciona la acción |
| | Anadir.java | Servlet para añadir un nuevo regalo |
| | Listar.java | Servlet para listar la bolsa de regalo |
| | Login.java | Servlet para la autenticación (inicio de sesión) |
| | Logout.java | Servlet para anular la sesión |

# Ejercicio 2.

- Re-estructuración de la web ReyesMagos.

| Tipo de archivo | Nombre | |
|---|---|---|
| css | WebContent/css/CSS.css | Css del ejercicio |
| html | Error_Contrasena.html | Página cuando se produce un error |
| | Inicio.html | Página principal del proyecto |
| | Cabecera.html | Genera la cabecera de la web |
| | Menu_No_Autenticado.html | Menu usado cuando no se esta autenticado |
| JSP | checkLogin.jsp | Comprueba si el usuario y contraseña son válid. Si es correcto → Introducir.jsp |
| | error.jsp | Usado para el control de errores |
| | Introducir.jsp | JSP usado para introducir el regalo. Se retorna así mismo (debe existir sesión) |
| | Listar.jsp | JSP para listar la bolsa de regalo (debe existir sesión) |
| | Login.jsp | JSP para la autenticación (inicio de sesión) |
| | Logout.jsp | JSP para anular la sesión |
| | MenuAutenticado.jsp | JSP que muestra un menu una vez autenticado (muestra el nombre del usuario) |

# Ejercicio 2.

## Login.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"
session="true"%>
<html>
  <head>
     <meta http-equiv="content-type" content="text/html; charset=utf8" />
     <title>Carta a los reyes Magos</title>
     <link rel="stylesheet" type="text/css" href="css/CSS.css" />
  </head>
<body>
   <%@ include file="Cabecera.html"%>
   <div id="contenedor">
      <%@ include file="Menu_No_Autenticado.html"%>
      <%
         if (request.getParameter("error") != null) {
         out.println(request.getParameter("error"));
         }
      %>
      <div id="Content">
         <form action="checkLogin.jsp" method="post">
            <label>Usuario: </label> <input type="text" name="user" id="user" /><br />
            <label>Password:</label> <input type="password" name="password"
                    id="password" /><br />
            <input type="submit" value="Validar" />
         </form>
      </div>
  </div>
</body>
</html>
```

Incluye html

Puede recibir un parámetro error → Intento anterior

Formulario

# Ejercicio2

## CheckLogin.jsp (Sólo jsp, pues no va a generar html)

```
String user = null, password = null;
if (request.getParameter("user") != null) user = request.getParameter("user");
if (request.getParameter("password") != null) password = request.getParameter("password");
if (user == null || password == null) {
  %>
  <jsp:forward page="Login.jsp"><jsp:param name="error" value="Usuario y/o
claveincorrectos.<br>Vuelve a intentarlo." /></jsp:forward>
<%
} else{
  Connection con =null;ResultSet resultados=null;
  try{
                Context ctx = new InitialContext();
                DataSource ds = (DataSource) ctx.lookup("java:comp/env/jdbc/testdb");
                con = ds.getConnection();
                String query = "SELECT * FROM usuarios where username=? and password=?";
                PreparedStatement sentencia = con.prepareStatement(query);
                sentencia.setString(1, user);sentencia.setString(2, password);
                 resultados = sentencia.executeQuery();
                if (resultados.next() == false) {
                %>
                   <jsp:forward page="Login.jsp"><jsp:param name="error"
                  value="Usuario y/o claveincorrectos.<br>Vuelve a intentarlo." /></jsp:forward>
                <%
                } else {
                                session.setAttribute("user", user);
                                session.setAttribute("id",resultados.getString("id"));
                                con.close();%>
                                <jsp:forward page="Introducir.jsp" />
                <%}
  }catch (SQLException e) {%>
                <jsp:forward page="Login.jsp"><jsp:param name="error" value="Usuario y/o
                claveincorrectos.<br>Vuelve a intentarlo." /></jsp:forward>
<%} } %>
```

Si no hay valores → Login.jsp con error

Pool de Conexiones

Consulta con PreparedStatement

Si es correcto → Introducir.jsp

Si no es correcto → Login.jsp con error

# Ejercicio2

## Introducir.jsp

Session

Se comprueba si hay sesion

Incluir info con jsp:include

Formulario

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"
session="true"%>
<html><head>
    <title>Carta a los reyes Magos</title>
    <link rel="stylesheet" type="text/css" href="css/CSS.css" />
</head> <body>
<%
  String user = (String) session.getAttribute("user");
  if (user==null) {

response.sendRedirect(response.encodeRedirectURL("error_contrasena.html"));
  } %>
  <%@ include file="Cabecera.html"%>
  <div id="contenedor">
     <jsp:include page="Menu_Autenticado.jsp">
          <jsp:param value="user" name="$user"/>
     </jsp:include>
     <div id="Content">
         <form method="post" action="Anadir.jsp" name="datos">
          Regalo: <input name="Regalo" id="Regalo"> <br>
                  <button>Añadir regalo a la carta</button>
         </form>
         <form method="post" action="Action.jsp" name="datos">
          <button>Listar Regalos</button>
         </form>
    </div>
  </div>
</body></html>
```

# Ejercicio2

- Logout.jsp (Sólo jsp, pues no va a generar html)

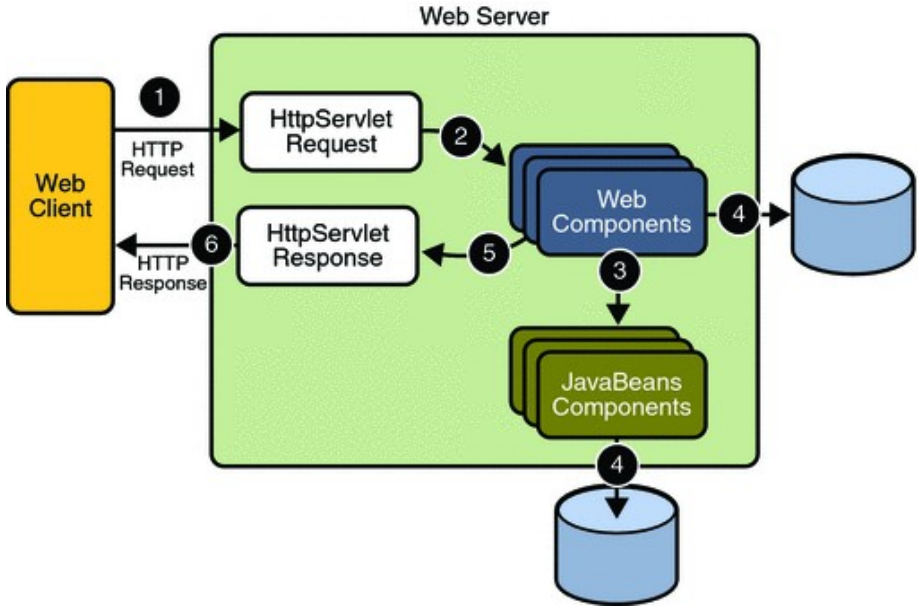
```
<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8" session="true"%>
<%
session.invalidate();
%>
<jsp:forward page="Login.jsp"/>
```

- Menu_Autenticado.jsp (Recibe el user)

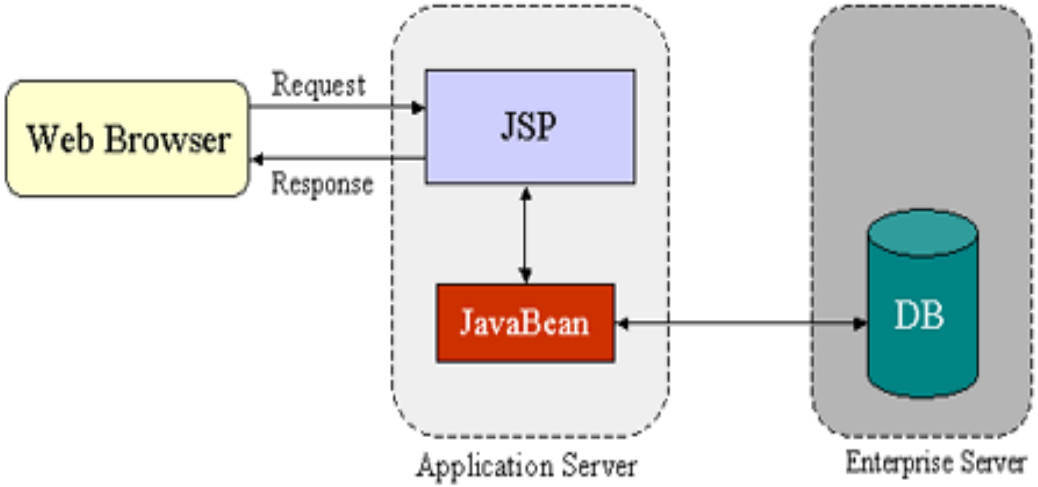
```
<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8"%>
<div id="Menu">
   <a href="Autenticar.html" title="Entrada">Login</a><br/>
   <a href='Logout.jsp' title=''>Logout</a><br />
         Usuario: <%= session.getAttribute("user")%>
</div>
```

# 6. Modelo Vista Controlador

- Modelo Servlet

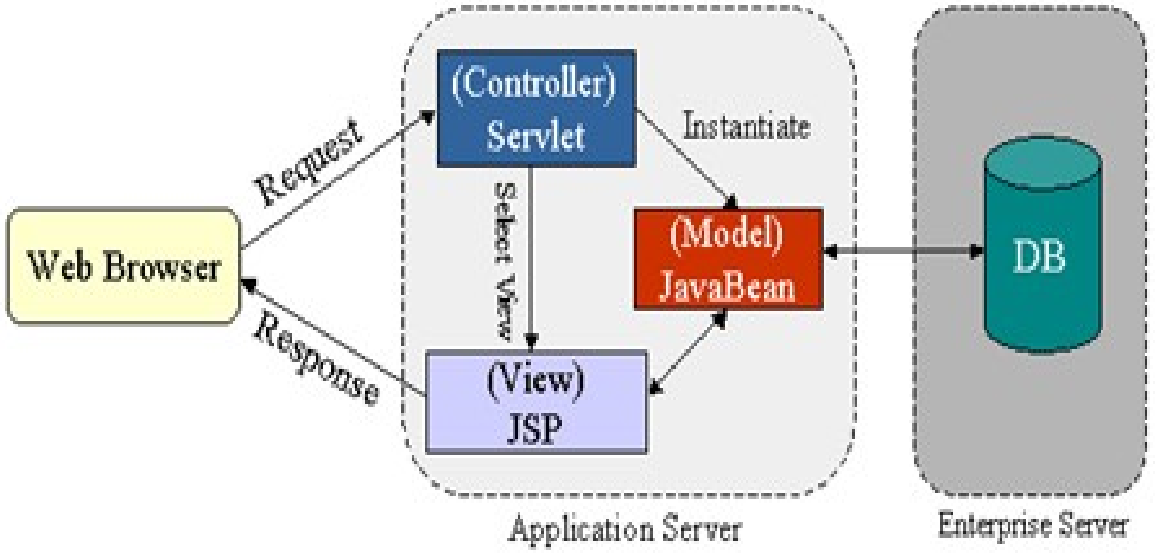



- Modelo JSP



# 6. Modelo Vista Controlador

- Modelo Vista Controlador
  - Modelo: se encarga de representar los objetos del dominio y contener la lógica de negocio (Beans)
  - Vista: se encarga de presentar los datos al usuario (JSP)
  - Controlador: acepta las acciones del usuario, las convierte en manipulaciones sobre el modelo y selecciona vistas adecuadas para representar la información relacionada con las acciones del usuario (Servlet)



# Bibliografía


- Tutorial básico de java EE. Abraham Otero. JavaHispano
  - http://www.javahispano.org/portada/2011/7/28/tutorial-basico-de-java-ee-por-abraham-otero.html
- Tutorial de Servlets de Sun:
  - http://java.sun.com/j2ee/tutorial/1_3-fcs/doc/Servlets.html
- Libro Gratuito sobre Servlets y JSP (muy completo):
  - http://www.theserverside.com/tt/books/addisonwesley/ServletsJSP/ind
- Sitio web dedicado a la programación con servlets y JSP:
  - http://www.moreservlets.com/
- Ejemplos
  - http://www.java2s.com/Tutorial/Java/0400__Servlet/Catalog0400__Servlet.htm